SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name:B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s)Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju Mr. G. Kranthi		
Course Code	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/I	**Regulation**	R23
Date and Day of Assignment	Week 6 - Thursday	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All Batches
AssignmentNumber:12.4 (Present assignment number)/**24**(Total number of assignments)			

	Question	Expected Time to complete
1	**Lab 12 – Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms** **Lab Objectives** • Apply AI-assisted programming to implement and optimize sorting and searching algorithms. • Compare different algorithms in terms of efficiency and use cases. • Understand how AI tools can suggest optimized code and complexity improvements. **Learning Outcome** After completing this assignment, students will be able to: • Implement classical algorithms with AI assistance • Compare algorithm efficiency using real-world scenarios • Understand when optimization is necessary • Critically evaluate AI-generated suggestions instead of blindly accepting them	Week 6

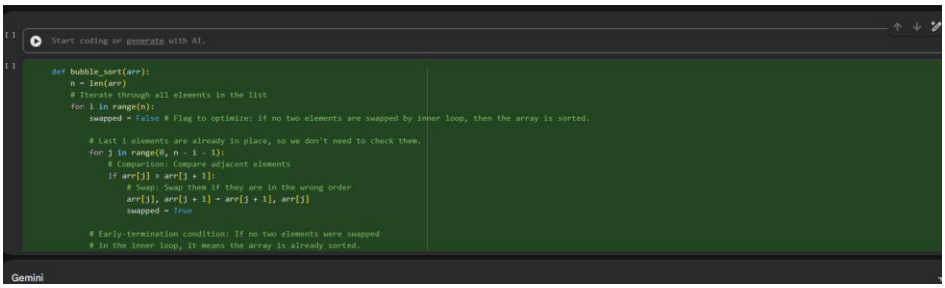
	Task 1: Bubble Sort for Ranking Exam Scores
	
	Scenario
	
	You are working on a **college result processing system** where a small list of student scores needs to be sorted after every internal assessment.
	
	Task Description
	
	• Implement **Bubble Sort** in Python to sort a list of student scores.
	• Use an AI tool to:
	  ○ Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes
	  ○ Identify early-termination conditions when the list becomes sorted	

- Provide a brief time complexity analysis

Expected Outcome

- A Bubble Sort implementation with:
  - AI-generated comments explaining the logic
  - Clear explanation of best, average, and worst-case complexity
  - Sample input/output showing sorted scores

Visualize the Bubble Sort algorithm's execution step-by-step on the given `student_scores` list, highlighting each comparison and swap operation." This would involve showing the state of the list at each significant step.



```
print(f"Sorted scores: {sorted_scores}")
```

Time Complexity Analysis of Bubble Sort

- **Best Case: O(n)**
  - This occurs when the array is already sorted. The outer loop runs `n` times, but the `swapped` flag will be `False` after the first pass of the inner loop (since no swaps are made). The algorithm will then break early, resulting in a single pass through the array. This makes the complexity linear, proportional to the number of elements `n`.
- **Average Case: O(n^2)**
  - In the average case, elements are in a random order. Each element might need to 'bubble up' to its correct position. The nested loops mean that for each of `n` elements, we might perform up to `n` comparisons and swaps, leading to `n * n = n^2` operations.
- **Worst Case: O(n^2)**
  - This happens when the array is sorted in reverse order. In this scenario, every comparison leads to a swap, and the largest

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Scenario

You are maintaining an **attendance system** where student roll numbers are already *almost sorted*, with only a few late updates.

Task Description

- Start with a Bubble Sort implementation.
- Ask AI to:
  - Review the problem and suggest a more suitable sorting algorithm
  - Generate an **Insertion Sort** implementation
  - Explain why Insertion Sort performs better on nearly sorted data
- Compare execution behavior on nearly sorted input

Expected Outcome

- Two sorting implementations:
  - Bubble Sort
  - Insertion Sort
- AI-assisted explanation highlighting efficiency differences for partially sorted datasets

- Prompt
  use sorting implementations

  Bubble Sort, Insertion Sort

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

student_scores = [64, 34, 25, 12, 22, 11, 90]
print(f"Original scores (Bubble Sort): {student_scores}")
sorted_scores_bubble = bubble_sort(list(student_scores)) # Use a copy to avoid modifying original list
print(f"Sorted scores (Bubble Sort): {sorted_scores_bubble}")
```

```python
def insertion_sort(arr):
    # Traverse through 1 to len(arr)
    for i in range(1, len(arr)):
        key = arr[i] # Element to be inserted into the so

        # Move elements of arr[0..i-1], that are greater
        # to one position ahead of their current position
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j] # Shift element to the ri
            j -= 1
        arr[j + 1] = key # Place key in its correct posit
    return arr

# Sample input list of student scores
student_scores_insertion = [64, 34, 25, 12, 22, 11, 90]
```

assisted explanation highlighting efficiency differences for partially sorted datasets

Let's compare Bubble Sort and Insertion Sort. I'll start by providing the Bubble Sort implementation, then generate the Insertion Sort code, and finally, explain why Insertion Sort is often more suitable for nearly sorted data, including a comparison of their execution behavior.

Accept & Run | Accept | Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

AI ASST 12.4.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

- `12, 15` (no swap)
- `15, 8` (swap) -> `[10, 12, 8, 15, 20]`
- `15, 20` (no swap)
- `swapped` is `True`.

- **Pass 2:**
  - `10, 12` (no swap)
  - `12, 8` (swap) -> `[10, 8, 12, 15, 20]`
  - `12, 15` (no swap)
  - `15, 20` (no swap)
  - `swapped` is `True`.
- **Pass 3:**
  - `10, 8` (swap) -> `[8, 10, 12, 15, 20]`
  - `10, 12` (no swap)
  - `12, 15` (no swap)
  - `15, 20` (no swap)
  - `swapped` is `True`.
- **Pass 4:**
  - `8, 10` (no swap)
  - ... (no swaps)
  - `swapped` is `False`, algorithm terminates.

As you can see, for the single misplaced element `8`, Insertion Sor
Sort, even with the early termination optimization, required multipl
position, performing more comparisons and swaps in total for the

assisted explanation highlighting efficiency differences for partially sorted datasets

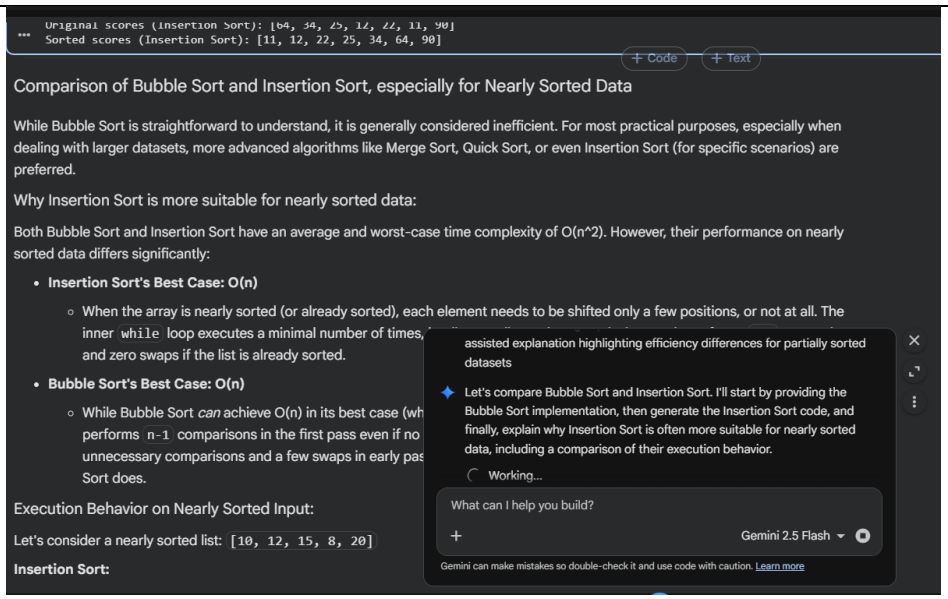
Let's compare Bubble Sort and Insertion Sort. I'll start by providing the Bubble Sort implementation, then generate the Insertion Sort code, and finally, explain why Insertion Sort is often more suitable for nearly sorted data, including a comparison of their execution behavior.

Implement Merge Sort and compare its efficiency | Time both sorting algorithms on a larger, r

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more



Task 3:

Searching Student Records in a Database

Scenario

You are developing a **student information portal** where users search for student records by roll number.

Task Description

- Implement:
  - **Linear Search** for unsorted student data
  - **Binary Search** for sorted student data
- Use AI to:
  - Add docstrings explaining parameters and return values
  - Explain when Binary Search is applicable
  - Highlight performance differences between the two searches

Expected Outcome

- Two working search implementations with docstrings
- AI-generated explanation of:
  - Time complexity

- Use cases for Linear vs Binary Search
- A short student observation comparing results on sorted vs unsorted lists

```python
def binary_search(data, target_roll_number):
    """
    Performs a binary search to find a student record by roll number.
    Requires the input 'data' list to be sorted by 'roll_number'.

    Args:
        data (list of dict): A list where each element is a dictionary representing a student record.
                             Each dictionary must contain a 'roll_number' key.
                             The list *must* be sorted in ascending order by 'roll_number'.
        target_roll_number (int): The roll number to search for.

    Returns:
        dict or None: The student record dictionary if found, otherwise None.
    """
    low = 0
    high = len(data) - 1

    while low <= high:
        mid = (low + high) // 2
        current_roll_number = data[mid].get('roll_number'

        if current_roll_number == target_roll_number:
            return data[mid]
        elif current_roll_number < target_roll_number:
            low = mid + 1
        else:
            high = mid - 1
    return None

# Sample student data, sorted by roll_number for Binary S
students_sorted = sorted([
    {'name': 'Bob', 'roll_number': 101, 'grade': 'B'},
```

Scenario You are developing a student information portal where users search for student records by roll number. Task Description Implement: Linear Search for unsorted student data Binary Search for sorted student data Use AI to: Add docstrings explaining parameters and return values Explain when Binary Search is applicable Highlight performance differences

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Gemini

Explanation: Linear Search vs. Binary Search

When is Binary Search applicable?

Binary Search is a highly efficient searching algorithm, but it has a crucial prerequisite: the data structure (e.g., a list or array) must be **sorted** in ascending or descending order. If the data is not sorted, Binary Search will not work correctly and may return incorrect results or fail to find existing elements.

Performance Differences (Time Complexity)

- **Linear Search:**
  - **Time Complexity:** O(n)
  - **Explanation:** In the worst-case scenario (target element is at the end or not present), Linear Search has to check every element in the list. This means the number of operations grows linearly with the size of the list (n). For an unsorted list, this is the only option if you cannot modify the data.
- **Binary Search:**
  - **Time Complexity:** O(log n)
  - **Explanation:** Binary Search works by repeatedly dividi in each step. This logarithmic growth means that even t small. For example, to search a list of 1 million elements Search would take at most around 20 comparisons (log

Use Cases for Linear vs. Binary Search

- **Linear Search is preferred when:**
  - The data is unsorted, and sorting the data is too costly

Scenario You are developing a student information portal where users search for student records by roll number. Task Description Implement: Linear Search for unsorted student data Binary Search for sorted student data Use AI to: Add docstrings explaining parameters and return values Explain when Binary Search is applicable Highlight performance differences

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Linear Search and Binary Search for student records. I'll provide the code with docstrings, and then explain their differences in applicability and performance, especially when dealing with sorted vs. unsorted data.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario

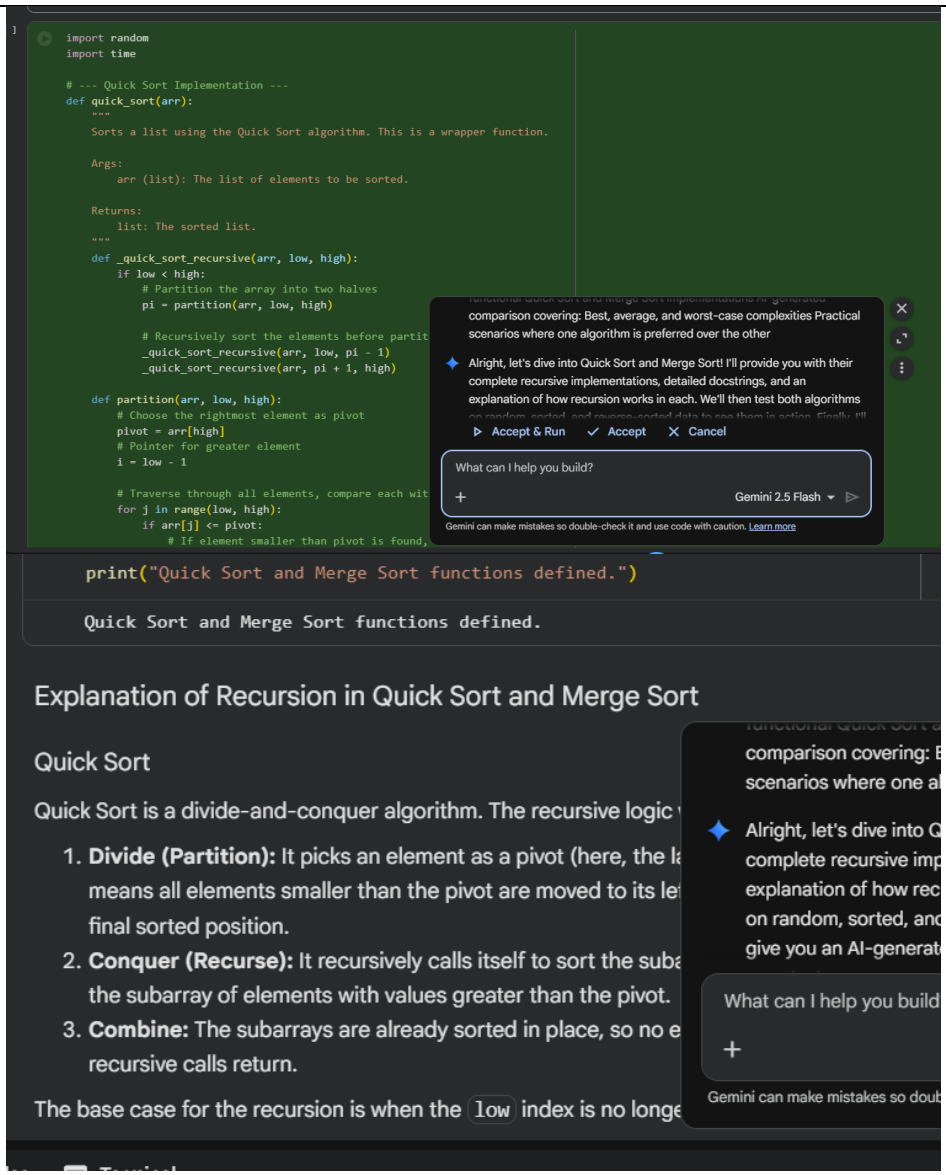
You are part of a **data analytics team** that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Task Description

- Provide AI with partially written recursive functions for:
  - **Quick Sort**
  - **Merge Sort**
- Ask AI to:
  - Complete the recursive logic
  - Add meaningful docstrings
  - Explain how recursion works in each algorithm
- Test both algorithms on:
  - Random data
  - Sorted data
  - Reverse-sorted data

Expected Outcome

- Fully functional Quick Sort and Merge Sort implementations
- AI-generated comparison covering:
  - Best, average, and worst-case complexities
  - Practical scenarios where one algorithm is preferred over the other



Task 5: Optimizing a Duplicate Detection Algorithm

Scenario

You are building a **data validation module** that must detect duplicate user IDs in a large dataset before importing it into a system.

Task Description

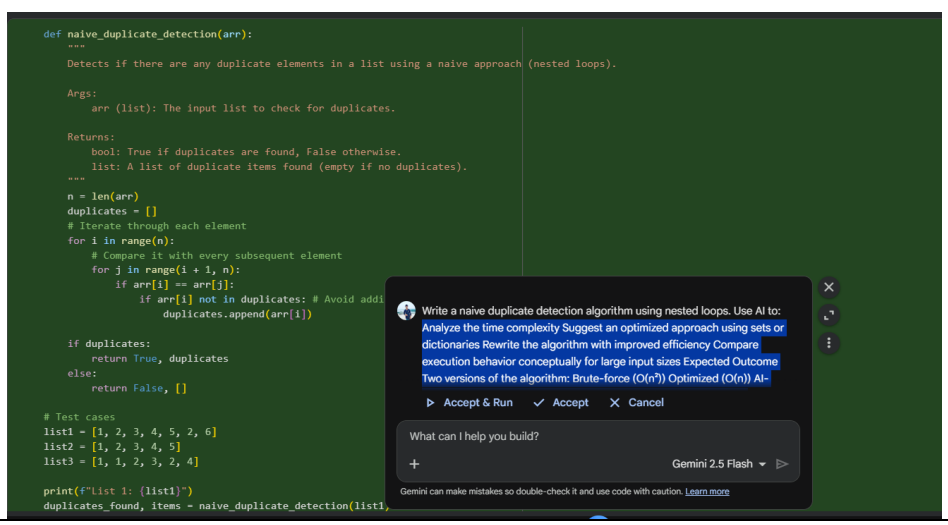
- Write a **naive duplicate detection algorithm** using nested loops.
- Use AI to:
  - Analyze the time complexity
  - Suggest an optimized approach using sets or dictionaries
  - Rewrite the algorithm with improved efficiency
- Compare execution behavior conceptually for large input sizes

Expected Outcome

- Two versions of the algorithm:
  - Brute-force ($O(n^2)$)
  - Optimized ($O(n)$)
- AI-assisted explanation showing how and why performance improved

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Analyze the time complexity Suggest an optimized approach using sets or dictionaries Rewrite the algorithm with improved efficiency Compare execution behavior conceptually for large input sizes Expected Outcome Two versions of the algorithm: Brute-force ($O(n^2)$) Optimized ($O(n)$) AI-assisted explanation showing how and why performance improved



AI ASST 12.4.ipynb

File Edit View Insert Runtime Tools Help

ommands + Code + Text Run all

```
duplicates_found, items = naive_duplicate_detection(list3)
print(f"Naive Detection - Duplicates found: {duplicates_found}, Items: {items}")
```

+ Code + Text

Time Complexity Analysis of Naive Duplicate Detection (Nested Loops)

Time Complexity: $O(n^2)$

- **Explanation:** The naive duplicate detection algorithm uses two nested loops. The outer loop iterates through each element of the list (from `0` to `n-1`). For each element in the outer loop, the inner loop iterates through the remaining elements (from `i+1` to `n-1`) to compare them. In the worst-case scenario (e.g., no duplicates, or duplicates at the very end), almost every element is compared with every other element once. This results in approximately `n * (n-1) / 2` comparisons, which simplifies to a quadratic time complexity, $O(n^2)$.
- **Impact on large input sizes:** For a list of `n` elements, if `n` doubles, the number of operations roughly quadruples. This makes the algorithm very inefficient for large datasets, as the execution time grows rapidly with the input size.

Suggestion for an Optimized Approach

The quadratic time complexity of the naive approach is often unac
duplicates is to use a hash-based data structure, such as a **set** or

- **Using a Set:** Sets in Python are collections of unique elemen
set, we can detect duplicates. If an element is already in the
(`element in set`) and adding elements (`set.add(elemen`

This approach can reduce the time complexity significantly from C

```
def optimized_duplicate_detection(arr):
    """
```

Write a naive duplicate detection algorithm using nested loops. Use AI to: Analyze the time complexity Suggest an optimized approach using sets or dictionaries Rewrite the algorithm with improved efficiency Compare execution behavior conceptually for large input sizes Expected Outcome Two versions of the algorithm: Brute-force ($O(n^2)$) Optimized ($O(n)$) AI-assisted explanation showing how and why performance improved

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

AI ASST 12.4.ipynb

File Edit View Insert Runtime Tools Help

mmands + Code + Text Run all

- **Why it's good for large inputs:** The execution time grows linearly with the input size. If `n` doubles, the execution time roughly doubles. This makes it highly scalable. For a list of millions of items, this algorithm would typically complete in a matter of seconds or minutes, a stark contrast to the naive approach.

How and Why Performance Improved:

The performance improvement stems directly from the choice of data structure and the underlying operations:

1. **Reduced Comparisons:** The naive approach requires comparing each element with potentially every other element. The optimized approach, by using a set, converts the comparison task into a fast lookup operation. Instead of iterating through a sub-list, it performs an $O(1)$ average-time hash lookup.
2. **Efficient Lookups (Hashing):** Sets (and dictionaries) in Python are implemented using hash tables. Hash tables provide nearly constant-time average performance for adding, removing, and checking for the presence of elements. This avoids the need for repeated linear scans that characterize the naive approach.
3. **Single Pass vs. Nested Passes:** The optimized algorithm typically requires only a single pass through the list to build up the `seen` set and identify duplicates. The naive algorithm, by contrast,
comparisons (the inner loop).

In summary, the optimized approach leverages the efficiency of ha
complexity from quadratic ($O(n^2)$) to linear ($O(n)$), resulting in mas

Write a naive duplicate detection algorithm using nested loops. Use AI to: Analyze the time complexity Suggest an optimized approach using sets or dictionaries Rewrite the algorithm with improved efficiency Compare execution behavior conceptually for large input sizes Expected Outcome Two versions of the algorithm: Brute-force ($O(n^2)$) Optimized ($O(n)$) AI-assisted explanation showing how and why performance improved